Link Layer Latency

Day 1: Network and OS Foundations for Safe Robotic Actuation

Mentor - Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors - Mr. Jiang Suyu, Mr. Ruan Haozhe

July 2026
Summer Course at Lanzhou University
The Hong Kong Polytechnic University (PolyU)

Table of content

This Week

One question, five days.

What creates delay or unsafety between a *plan* and an *actuator*, and how do network and OS primitives bound it?

Day	Lecture	Project hands-on (60 min)
1	Link layer latency	State contract + FrameV1 integrity
2	Application layer latency	MQTT, timestamps + freshness gate
3	OS: network management	Nonblocking multi-client `poll()`
4	OS: I/O locking	Single writer + safety audit
5	OS: sync & async + capstone	Preemption + evidence-backed release

Everything in C, everything measurable.

Each lab produces numbers (error detection rates, latencies, throughput) that you collect yourself.

1 A Plan, a Wire, an Arm

The Hair-Cutting Robot Problem

The setting.
An industrial hair-cutting robot: a planner process decides *cut here, move there*; a robot arm executes. Scissors move next to a human head.

Between plan and arm there is a wire.
The planner's intent must cross a physical link, in our lab a serial port, before anything moves.

The safety invariant of this whole week.
The arm must *never* act on a command it did not receive *correctly* and *in time*.

Two Enemies: Delay and Corruption

Delay.
A stop command that arrives 800 ms late is not a stop command. Every stage (queueing, transmission, processing) eats into a fixed *latency budget*.

Corruption.
Noise can flip bits in flight. A corrupted `MOVE(3 cm)` that decodes as `MOVE(30 cm)` is worse than a lost message.

Design consequence.
Losing a frame is recoverable (retransmit). Acting on a wrong frame is not. We therefore *detect and reject*, never guess.

Your Deliverable: A Wire Contract

There is no vendor SDK in this course.
You design the wire protocol yourselves, today: how a command becomes bytes, and how the receiver knows those bytes are intact.

The frame spec is the deliverable.
Sync byte, length, command, sequence number, CRC, written down as a one-page contract. Your team reuses it every day this week.

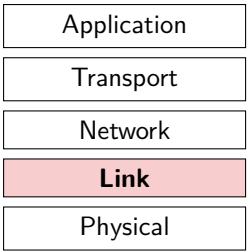
Why this matters beyond the course.
Real robot platforms work the same way: a published command schema is the contract between planning software and actuation hardware.

2 What the Link Layer Does

Where the Link Layer Sits

One hop, not end to end.

Application, transport and network layers think in *end-to-end* terms. The link layer moves a frame across *one physical hop*: PC to arm, host to router.



Today's scope.

We deliberately stay on this one hop. Days 2 and 3 climb back up the stack.

Link-Layer Services

Framing.
Wrap payload bits into frames with recognisable boundaries; the receiver must know where a message starts and ends.

Error detection (and sometimes correction).
Add redundant bits so the receiver can tell a damaged frame from a healthy one: parity, checksums, CRC.

Link access and reliable delivery.
Shared media need access rules (Ethernet, Wi-Fi); some links also retransmit locally. Our point-to-point serial line needs neither, which is exactly why *we* must supply integrity ourselves.

Our Link: a Raw Serial Byte Pipe

UART serial, 115200 baud, 8N1.
Each byte travels as 10 bits on the wire (start + 8 data + stop). Effective rate: about 11 520 bytes per second.

What the serial port gives you.
A stream of bytes, in order, one hop. Nothing else.

What it does *not* give you.
No message boundaries, no integrity check, no acknowledgement. Ethernet's adapter does framing and CRC in hardware; on a raw UART, you are the link layer.

3 The Anatomy of Delay

Four Sources of Delay

Every hop charges four tolls.

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

Processing: examine the frame, validate, decide where it goes. Microseconds, but Day 3 shows how a busy OS stretches it.

Queueing: waiting behind other frames for the link. Zero to unbounded; the only one that *grows with load*.

Transmission: pushing the bits onto the wire, $d_{\text{trans}} = L/R$ for L bits at rate R.

Propagation: bits travelling the physical medium, $d_{\text{prop}} = d/s$ for distance d at speed s.

Transmission vs Propagation

The classic confusion.
Transmission delay is how long the *toll booth* takes to push the cars through; propagation is the *highway* time to the next booth.

They scale differently.
Double the frame size: transmission doubles, propagation unchanged. Double the cable length: propagation doubles, transmission unchanged.

Rule of thumb for short links.
On a 2 m serial cable, propagation is about 10 ns. On slow links over short distances, transmission dominates: the wire speed, not the wire length, is what you feel.

Worked Numbers on Our Lab Link

One 16-byte command frame at 115200 baud, 8N1.

$$d_{\text{trans}} = \frac{16 \times 10 \text{ bits}}{115200 \text{ bits/s}} \approx 1.39 \text{ ms}$$

Propagation over 2 m of cable.

$$d_{\text{prop}} = \frac{2 \text{ m}}{2 \times 10^8 \text{ m/s}} = 10 \text{ ns}$$

Reading.
Transmission is 10^5 times larger than propagation here. A 64-byte frame would already cost about 5.6 ms: frame size is a latency decision, not just a formatting one.

Queueing: the Delay That Grows

Traffic intensity.
With arrival rate a frames/s of L bits each on a link of rate R:

$$\text{intensity} = \frac{L \cdot a}{R}$$

Three regimes.
Intensity near 0: negligible queueing. Approaching 1: delay grows steeply. Greater than 1: the queue grows without bound, because frames arrive faster than the wire can drain them.

Robotics reading.
A queue of stale `MOVE` commands is a safety hazard: the arm executes the *past*. Day 5's e-stop lab is precisely about jumping such a queue.

A Latency Budget for an Emergency Stop

Think in budgets, not averages.
Suppose policy says: from operator's click to motor halt, at most 50 ms.

Stage	Day we measure it
Operator process → message published	Day 2 (MQTT)
Broker → bridge delivery	Day 2 (QoS comparison)
Bridge queue → serial transmission	Day 3 (socket queues), today (L/R)
Simulator parse → actuator halt	Day 5 (`epoll` e-stop)

Today's contribution.
You can already bound one line exactly: a frame of L bits can never beat L/R on the wire. Budgets are built from such hard lower bounds.

In-Class Quiz: How Slow Is the Wire?

At 115200 baud with 10 wire bits per byte, which is closest to the transmission delay of a 32-byte frame?

1. 0.28 ms
2. 1.4 ms
3. 2.8 ms
4. 28 ms

In-Class Quiz: Answer

Answer. $\boxed{3}$

Why.
$d_{\text{trans}} = (32 \times 10)/115200 \approx 2.8\,\text{ms}$.

The scaling to remember.
Delay is linear in frame size: option 2 is exactly the 16-byte frame from the worked example, and doubling the frame doubled the delay.

4 Framing a Byte Stream

Why Framing at All

The stream has no boundaries.
`read()` on a serial port may return half a command, or two and a half. The wire delivers bytes, not messages.

The receiver's question.
Given an endless byte stream, where does a command start, and where does it end?

Framing is the answer.
A convention, agreed by both sides in advance, that marks boundaries inside the stream. This convention is the first half of your wire contract.

Framing Strategies

Fixed length.
Every frame is exactly N bytes. Trivial to parse, wasteful for short commands, brittle if one byte is ever lost.

Length field.
A header byte announces the payload size. Compact and fast, but a corrupted length field desynchronises everything after it.

Sync byte + length + resynchronisation.
Start every frame with a known marker (e.g. `0xAA`); after any error, scan forward to the next marker and resume. This robust hybrid is what most binary protocols use, and what we recommend for your spec.

Sequence Numbers

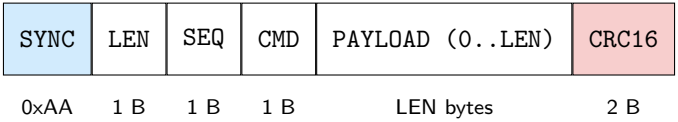
Why number the frames.
When a frame is rejected and retransmitted, the receiver must tell a *retry* from a *new command*: replaying a `MOVE` twice moves the arm twice.

One byte is enough here.
A counter that wraps at 255 lets the receiver detect duplicates and lets ACKs name exactly which frame they confirm.

Foreshadowing Day 2.
MQTT QoS levels wrestle with the same duplicate problem at the application layer. Same idea, one layer up.

A Candidate Frame Layout

One reasonable shape.
Yours may differ, but your spec must justify each field.



Design questions your spec sheet must answer.
What does CRC cover (header too, or payload only)? What if `0xAA` appears inside the payload? Maximum payload length, and what latency does that imply at 115200 baud?

The Frame in C, and a Serialization Warning

```c
typedef struct {
    uint8_t  sync;       /* always 0xAA             */
    uint8_t  len;        /* payload bytes, 0..32    */
    uint8_t  seq;        /* wraps at 255            */
    uint8_t  cmd;        /* MOVE, HOME, STOP, ...   */
    uint8_t  payload[32];
    uint16_t crc;        /* CRC-16 over len..payload */
} frame_t;
```

Never `write()` the struct itself.
The compiler may insert padding, and multi-byte fields have a byte order. Serialize field by field into a `uint8_t` buffer, and document the byte order in your spec.

5 Error Detection: Parity to CRC

Bit Errors Are Physical

Where flips come from.
Electrical noise, crosstalk, EMI from motors; a robot arm is itself a noise source sitting next to its own command cable.

The model.
Somewhere between sender and receiver, some bits invert. The receiver sees a perfectly plausible byte stream, just not the one that was sent.

In the lab.
Our *fault injector* sits between the two serial endpoints and flips bits at a rate you choose. You will watch your own protocol get attacked, live.

Detect, Then Refuse to Act

Error detection vs error correction.
Detection spends a few redundant bits to *notice* damage; correction spends many more to *repair* it. Correction suits one-way links (deep space, broadcast).

We have a back channel.
Our receiver can reply. So we choose the cheap side: detect, reject, and ask again. ACK when a frame verifies, NACK (or silence and a timeout) when it does not.

The safety framing.
Detection does not make the link reliable. It makes the link honest: the arm acts only on frames proven intact, and ignorance defaults to inaction.

Single-Bit Parity

The scheme.
Append one bit so the total number of 1s is even. Receiver recounts; an odd count means damage.

What it catches.
Any *odd* number of flipped bits.

What it misses.
Any *even* number: two flips cancel out and the frame verifies. Under bursty noise (exactly what motors produce), errors cluster, so this failure mode is common, not exotic. One bit of protection is not enough for actuation.

Two-Dimensional Parity

Arrange data in a grid; add parity per row and per column.

1	0	1	1	1
1	1	0	1	0
0	1	1	1	1
0	1	0	1	0

The win.
The highlighted bit arrived flipped (sent as 0): its row check *and* its column check now fail; the intersection *locates* the bit, so it can even be corrected.

The cost.
Many parity bits, and multi-bit bursts still slip through. A stepping stone, not a destination.

The Internet Checksum

The scheme.
Treat the data as 16-bit words, sum with ones'-complement arithmetic, send the complement. Receiver sums everything; all-1s means pass.

Why transport layers like it.
Cheap in software: a tight loop of adds. This is what TCP and UDP use.

Why it is weak.
Swap two 16-bit words: sum unchanged, error invisible. Offsetting flips in different words can cancel. Acceptable as a last-resort end-to-end check; *not* acceptable as the only guard on a noisy actuator link.

CRC: Error Detection as Polynomial Division

The idea.
Treat the d data bits as a polynomial over GF(2): coefficients 0 or 1, arithmetic without carries, where add and subtract are both XOR.

The scheme.
Sender and receiver agree on a generator G of $r+1$ bits. The sender appends r bits R chosen so that the whole $d+r$ bit string is exactly divisible by G:

$$R = \text{remainder}\left(\frac{D \cdot 2^r}{G}\right)$$

The check.
Receiver divides what arrives by G. Remainder zero: accept. Anything else: damage, reject. All the mathematics hides in one well-chosen constant G.

CRC by Hand: One Small Example

Data $D = 101110$, generator $G = 1001$ ($r = 3$): divide $101110\,000$ by G.

Long division by repeated XOR: at each step, XOR G under the leftmost remaining 1.

```
101110000
1001
---------
001010000
  1001
---------
000011000
    1001
---------
000001010
      1001
---------
000000011
```

Remainder $R = 011$: transmit $101110\,011$, and the receiver's division by G yields zero.

What CRC Guarantees

Burst errors.
A CRC with r check bits detects *every* burst of length at most r; for CRC-16, every burst up to 16 bits. This is the property that matters under motor noise.

Odd errors.
With $(x+1)$ as a factor of G, all odd numbers of bit flips are caught.

Longer bursts.
Slip through with probability about 2^{-r}, one in 65 536 for CRC-16. Compare: parity missed *half* of multi-bit errors. This is why Ethernet uses CRC-32 in hardware, and why our 16-byte frames get CRC-16.

CRC-16 in C

The division, eight bits at a time.

0x1021 encodes the generator G of CRC-16-CCITT.

```c
uint16_t crc16_ccitt(const uint8_t *p, size_t n)
{
    uint16_t crc = 0xFFFF;
    while (n--) {
        crc ^= (uint16_t)*p++ << 8;
        for (int b = 0; b < 8; b++)
            crc = (crc & 0x8000) ? (uint16_t)((crc << 1) ^ 0x1021)
                                 : (uint16_t)(crc << 1);
    }
    return crc;
}
```

CRC-16 in C (Cont'd)

Reading the loop.
The outer loop feeds one byte into the working remainder; the inner loop performs eight single-bit division steps, XORing G in whenever the top bit is 1. Exactly the long division from the worked example.

Three lines for your spec sheet.
The generator, the initial value (`0xFFFF` here), and which bytes the CRC covers. Both ends must agree on all three, or every frame will be rejected.

Closing the Loop: ACK and NACK

Receiver side.
Deframe, recompute CRC. Match: execute, reply `ACK(seq)`. Mismatch: discard, reply `NACK(seq)`, and the arm does not move.

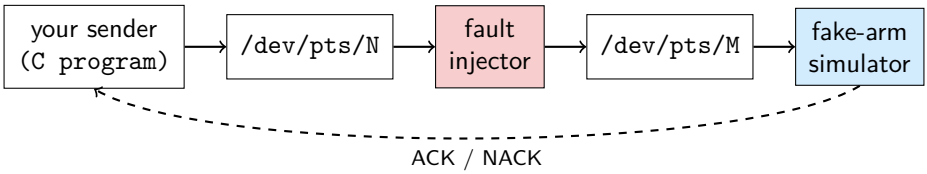
Sender side.
On NACK or timeout, retransmit with the *same* sequence number, up to a retry limit. The duplicate detection from your SEQ field earns its keep here.

What we have built.
A minimal reliable-delivery protocol over a raw byte pipe: the same ARQ pattern inside Wi-Fi links and TCP, sized down to one afternoon of C.

6 Project Hands-on Preview: Build the Link

The Hands-on Chain

No hardware required: the link is virtual, the noise is real.



The pieces we supply.

`socat` creates the two joined pseudo-terminals; the simulator renders an ASCII arm and answers ACK/NACK; the fault injector flips bits at a configurable rate. You write the sender.

Hands-on Tasks

Task 1: agree on a contract.
As a team, write the one-page frame spec: layout, byte order, CRC coverage, ACK/NACK rules, retry policy.

Task 2: implement it.
Framer, deframer with resync, and CRC-16, in the provided C scaffold; drive the simulated arm through a short command script.

Task 3: attack it.
Switch on the fault injector at bit-error rates 10^{-5} to 10^{-3}: how many frames are rejected, and do corrupted frames ever slip through CRC undetected?

Hands-on Tasks (Cont'd)

Pass condition.
Under active fault injection, the ASCII arm never once moves on a corrupted frame.

Keep what you build.
The frame spec and your framer/CRC code are not throwaway: every later lab this week speaks this protocol, and Day 2's bridge forwards exactly these frames.

Optional: the Same Code on Real Hardware

For groups with an Arduino servo arm.
Flash our reference firmware, then point your sender at `/dev/ttyUSB0` instead of the `socat` pseudo-terminal.

Zero code changes, only the device path.
If your program needs more than that, the protocol layering is leaky. A clean link layer should not know whether the far end is simulated.

This is a design principle, not a convenience.
Sim-before-real with a transport swap is exactly how the hair-cutting robot platform validates motion before scissors touch hair.

Day 1 Summary

Delay decomposes.
Processing + queueing + transmission + propagation; on our link, L/R dominates and frame size is a latency decision.

Framing turns streams into messages.
Sync byte, length, sequence number, plus a resync rule for when things go wrong.

CRC makes the link honest.
Bursts up to 16 bits always caught; the arm acts only on proven-intact frames.

Tomorrow.
Up the stack: publish/subscribe, MQTT QoS 0/1/2, and what each level costs in milliseconds.

Thanks for listening

Presented by Suyu Jiang

Template: Azusa Template